

DOSSIER TECHNIQUE DÉTAILLÉ

Comment Flowable a été intégré au Backend et à la Base de Données

1. Introduction au Cœur du Système (Backend)

Contrairement aux solutions BPM isolées (stand-alone), nous avons choisi une architecture dite "embarquée" (Embedded Engine). Le moteur Flowable est compilé et s'exécute directement à l'intérieur de notre Backend Java (Spring Boot). Cela permet une communication synchrone (sans latence réseau) entre la logique métier (IncidentFlow) et le moteur de processus (Flowable).

2. Configuration Maven (pom.xml)

L'intégration a débuté par l'ajout du "Starter" officiel dans le fichier pom.xml de Spring Boot. Ce starter configure automatiquement le moteur au démarrage de l'application.

backend/pom.xml

```
<dependency>
  <groupId>org.flowable</groupId>
  <artifactId>flowable-spring-boot-starter-process</artifactId>
  <version>7.0.1</version>
</dependency>
```

Grâce à cette dépendance, Spring injecte automatiquement les 4 services fondamentaux dans nos classes Java : RepositoryService, RuntimeService, TaskService, et HistoryService.

3. Modification de la Base de Données (PostgreSQL)

Lors du premier démarrage du Backend avec cette dépendance, Flowable a automatiquement exécuté des scripts SQL pour générer plus de 40 tables système dans la base IncidentFlow_db (ex: ACT_RU_EXECUTION, ACT_HI_TASKINST). C'est le schéma d'exécution.

Côté métier, notre table incidents a dû être modifiée pour créer une jointure logique vers Flowable :

Modification SQL (Entité Incident.java)

```
ALTER TABLE incidents ADD COLUMN process_instance_id VARCHAR(255);
```

Ce champ est la "clé étrangère conceptuelle". Il relie la ligne de l'incident (métier) à l'instance du workflow en cours (Flowable).

4. Déploiement Dynamique (Code Backend)

La page web permet d'envoyer un fichier XML (BPMN 2.0). Côté Backend, c'est le RepositoryService qui réceptionne ce fichier et l'inscrit dans la base de données (table ACT_RE_DEPLOYMENT) pour rendre ce workflow exécutable.

WorkflowController.java

```
@Autowired
private RepositoryService repositoryService;

public void deployProcess(MultipartFile file) {
    repositoryService.createDeployment()
        .addInputStream(file.getOriginalFilename(), file.getInputStream())
        .name("Incident Workflow")
        .deploy();
}
```

5. Le Cycle de Vie d'un Incident (Runtime & Task Services)

C'est ici que l'orchestration s'opère. Lorsqu'un utilisateur crée un ticket dans React, l'API Java intercepte la requête, sauvegarde la donnée, puis **délègue** le flux à Flowable :

IncidentService.java - Phase 1 : Démarrage

```
// 1. Le Backend démarre le processus BPMN (dont la clé est 'incidentWorkflow')
ProcessInstance instance = runtimeService.startProcessInstanceByKey("incidentWorkflow");

// 2. On lie la donnée métier au processus
incident.setProcessInstanceId(instance.getId());

// 3. On demande à Flowable : "Où en est le ticket ?"
Task currentTask = taskService.createTaskQuery()
    .processInstanceId(instance.getId())
    .singleResult();

// Le statut devient automatiquement le nom défini dans le schéma Modeler (ex: "Nouveau")
incident.setStatus(currentTask.getName());
```

Lorsqu'un opérateur valide une étape (bouton "Passer à En cours"), le code fait appel à la méthode `complete()` :

IncidentService.java - Phase 2 : Transition

```
// On ordonne au moteur de valider l'étape actuelle
taskService.complete(currentTask.getId());

// Flowable calcule automatiquement la prochaine étape selon le BPMN !
Task nextTask = taskService.createTaskQuery()
    .processInstanceId(incident.getProcessInstanceId())
    .singleResult();

if (nextTask != null) {
    incident.setStatus(nextTask.getName());
}
```

6. Conclusion et Traçabilité

Grâce à cette architecture fortement couplée, le code Java ne contient plus aucune condition de type `if (status == "Nouveau") then...`. Toute la logique est régie par la base de données Flowable. En parallèle, chaque appel à `startProcessInstanceByKey` ou `complete` déclenche automatiquement des inserts dans les tables d'audit (`ACT_HI_*`), constituant l'historique légal (`HistoryService`) sans ligne de code supplémentaire.

